Binding Agent Tool Calls to Effects: Counterfactually Validated Atoms for Pre-Commit Mediation

Anonymous Authors

Abstract

Tool-using language-model agents turn untrusted text into structured calls that commit external state. A single call can create an object, disclose data to several principals, and change visibility. A monitor that sees only the tool name, call, or raw arguments may therefore merge state changes that require different authorization decisions. We study *tool-effect binding*: the representation needed to connect a proposed call to the security-relevant effects it will commit. We show that when a representation merges effects that an admissible policy distinguishes, every monitor over that representation must either allow an unauthorized effect or withhold authorized work.

We introduce counterfactual registration, which treats a tool descriptor as a hypothesis, executes controlled call and state variants, and refines the descriptor when the observed effects expose a missing distinction. Source execution determines what changed; a separate policy oracle determines whether the change must be authorized independently. A common-field representation leaves 118 authorization-separating pairs across 56 source-executed calls, whereas typed effect atoms remove every observed collision. The same representation removes all observed collisions in 32 held-out ToolSandbox contexts and exactly realizes their frozen 232-query policy relation.

Finally, we use a conservative registered descriptor in a deterministic provenance-origin monitor. On 608 scope-aligned AgentDojo attacks under DeepSeek, attack success falls from 1.0% to 0.0%, and every executed call has a matching pre-commit decision. A same-model prompting defense reaches the same attack rate, while benign non-inferiority is not established. We therefore treat this runtime experiment as a bounded mediation case study; the main representation claim rests on source-executed collisions and the frozen finite-policy test.

1 Introduction

A user asks an agent to create a meeting with Alice. After reading an untrusted document, the agent calls the calendar tool with both Alice and Eve as participants. The call performs the requested work—creating the meeting—but also commits an additional invitation. This is a common shape of indirect prompt injection: content returned by one tool steers a later call toward an attacker-chosen effect [8, 26]. Once the invitation is emitted, later model reasoning cannot turn that committed disclosure back into a harmless proposal.

At the application boundary, both calls invoke `create_event`. Treating the whole call as one authorization unit forces the event creation and every invitation to share a decision. Inspecting raw arguments exposes Alice and Eve but does not identify which state transitions those values control; defaults and pre-state can also change the realized effect without changing an explicit argument. Provenance answers where a value came from, not what the tool will do with it. The underlying problem is therefore representational: a monitor cannot authorize a distinction that its view of the call does not preserve.

Figure 1 illustrates this collision. If two executions have the same monitor representation but some admissible policy must decide them differently, the monitor must either admit an unauthorized effect or withhold authorized work. Complete mediation and least privilege do not recover the missing distinction; they require an adequate object of mediation. We call this requirement *tool-effect binding*.

This observation separates four questions that are often conflated. First, did an execution change external state? Second, can an admissible policy require different decisions for two such changes? Third, does the monitor's representation preserve that distinction? Finally, does the runtime check the same call that it commits? State-difference evidence answers the first question, a policy-separating witness answers the second, representation collisions answer the third, and a pre-commit audit answers the fourth. No one layer substitutes for another.

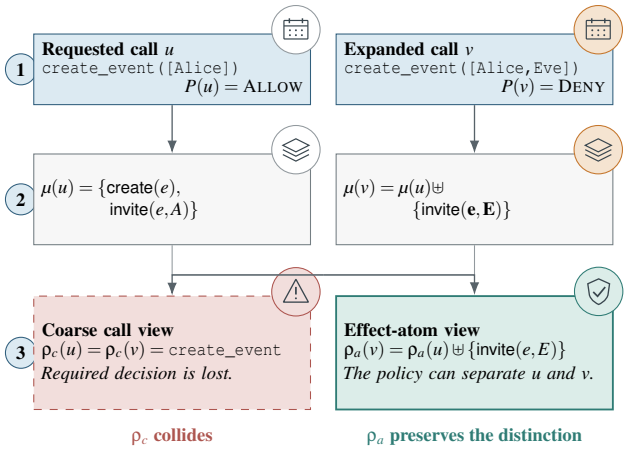

Figure 1: Why the authorization object matters. Calls u and v use the same tool, but source execution shows that v adds an invitation that the policy rejects. A coarse call view merges the pair; an effect-atom view exposes the additional committed effect before execution.

We represent a call as a set of concrete, independently authorizable *effect atoms*. In the example, these include creating the event, inviting Alice, and inviting Eve. An atom records the effect kind and the tool-local resource, target, operation, commit mode, provenance, or qualifier that an admissible policy may need to distinguish. A representation is minimal relative to a frozen tool domain and policy family when merging any two of its classes would erase a required authorization distinction.

The central challenge is validating this representation. A developer or an LLM may propose an effect inventory from a tool's description and schema, but the proposal alone says nothing about whether it matches the implementation. We therefore treat every candidate descriptor as an untrusted hypothesis. Before registration, a harness executes a valid call and controlled variants that change a field, default, target expansion, or pre-state. A source-state oracle compares the committed effects. If the effects change while the candidate representation does not, the execution supplies a counterexample and the descriptor is refined. Proposal provides a starting point; execution provides the evidence. Unresolved tests are retained instead of being interpreted as invariance.

This use of counterfactuals differs from defenses that rerun an agent to ask which observation caused a proposed action [14, 18]. We hold the planner out of the validation loop and intervene on the tool call or its pre-state to ask which effects the implementation commits. The two questions are complementary: causal attribution can constrain why a call was proposed, while effect registration determines what must be mediated if that call is about to execute.

After validation, the descriptor is hash-frozen into the tool interface. The same trust boundary applies online: an agent proposes a call but cannot authorize it. Immediately before commitment, a deterministic monitor expands the exact call through the registered descriptor and passes that effect view to an independently supplied authority predicate. Only the checked call associated with an ALLOW decision can execute. The registration LLM is absent from this runtime path, and the runtime planner cannot enlarge its own authority. Our AgentDojo prototype instantiates only provenance-origin confinement over registered fields and effect labels; richer ACL, delegation, and quota policies can consume a concrete atom view but are not implemented here.

Our evaluation follows the same sequence. First, source replay establishes that calls commit compound and pre-state-dependent effects. Second, policy-separating pairs show that common call views lose required distinctions, while typed atoms preserve them in two finite source-executed domains. Third, a concrete authorizer exactly realizes a frozen policy relation over those atoms. Finally, an AgentDojo provenance-origin monitor demonstrates audited pre-commit mediation but does not outperform a prompting defense or a raw-field control on utility. The first three results support the representation claim; the last exposes the current runtime tradeoff rather than hiding it.

This paper makes three contributions:

- **Model.** We formalize authorization-sufficient effect representations and prove the unavoidable safety–permission tradeoff caused by a representation collision.
- **Validate.** We introduce source-executed counterfactual registration, in which execution counterexamples refine tool-local effect atoms before a descriptor is frozen.
- **Instantiate.** We realize the interface in a finite concrete-atom authorizer and an audited provenance-origin monitor, separating the representation result from the narrower runtime policy it enables.

2 Related Work

Where defenses intervene. AgentDojo, ToolEmu, and AgentLAB expose indirect prompt injection in tool-rich and long-horizon tasks [8, 17, 26]. Step classifiers and prompt defenses decide whether a proposed action appears safe [15, 23]; CaMeL and IPIGuard preserve trusted control or track data dependencies [1, 7]. AttriGuard and CausalArmor go further by intervening on observations and rerunning the agent to estimate which input caused a proposed call [14, 18]. These approaches reason upstream about why the planner selected an action. Our registration procedure intervenes downstream on arguments, defaults, and pre-state, executes the tool, and asks which committed effects must remain distinguishable. It produces an object for authorization rather than another classifier of the planner's choice.

What authorization sees. Agent policy systems bound tools, actions, parameters, or capabilities [21, 29, 33–35]. PACT assigns semantic roles to arguments, while AuthGraph compares clean authority with the provenance of argument values [11, 32]. SecureClaw and commit-time authorization emphasize checking fresh authority at the effect sink [22, 28]. These systems are complementary to our representation question. An argument need not correspond one-to-one with an effect: defaults and pre-state can create effects without an explicit value, and one list argument can expand into several independently authorizable changes. We supply validated effect objects to a policy; we do not infer the user's authority. Contextual frameworks separately formalize task, action, source, and data-isolation boundaries [30]. Our enforcement boundary follows established complete mediation, least privilege, confused-deputy, information-flow, and provenance principles [4, 5, 9, 10, 13, 27].

How effects are specified. Type-and-effect systems make program effects explicit by construction [20]; ETAS brings effect rows and compiled monitors to an agent language [31]. Contract2Tool infers planning contracts, ContractGuard protects supplied contracts, and ToolGuardian combines descriptions, traces, mock execution, and source analysis to vet tools [2, 16, 25]. We address a narrower obligation that remains after an effect specification is proposed: does the representation preserve every distinction required by the selected policy family? Paired executions can falsify that property for existing tools that were not written in an effect-typed language.

How representations are validated. Counterexample-guided abstraction refinement starts with a coarse abstraction and splits it when a concrete execution falsifies the abstraction [6]. Causal abstraction similarly asks whether interventions at a lower level are preserved by a higher-level model [3, 12, 24]. We adapt this idea to tool registration: a call or state intervention is useful only when execution changes the committed effect, and it requires a representational split only when the policy also separates the pair. The conclusion is scoped to the frozen intervention and policy domains, not unrestricted causal discovery.

3 Problem and Security Model

System and enforcement boundary. A user gives an agent an authenticated task q. The agent may read messages, documents, webpages, or database records before proposing a structured tool call $c = (t, \bar{x})$. The LLM is an untrusted control-plane component: its plan, effect description, and call arguments are proposals rather than authority. Executing the fully instantiated call from pre-state s may commit several externally visible effects, such as creating an event, inviting principals, or changing its visibility. The enforcement point lies after argument totalization but immediately before those effects commit. The executor may commit only the exact call associated with the monitor's decision.

Threat model. The adversary controls content returned by one or more untrusted reads. It may embed instructions or attacker-chosen values, induce the agent to add a resource, target, payload, operation, or effect, and carry this influence across a multi-step trajectory. We assume the agent may follow the injected instruction completely; security does not rely on model alignment. The adversary cannot modify the authenticated task, forge trusted provenance, change the registered descriptor or policy, compromise the monitor, or replace a checked call before execution. It also cannot modify the tool implementation after registration. The trusted harness marks untrusted-content boundaries; the AgentDojo adapter derives typed provenance from those boundaries.

Security objective. At the architectural level, let $u = (t, \bar{x}, s, h)$ denote an execution context containing a tool, totalized arguments, pre-state, and typed provenance evidence. Let $\mu(u)$ denote its concrete committed effects. A policy context P may additionally depend on trusted execution metadata $\gamma(u)$, including effect provenance. Writing $\omega(u) = (\mu(u), \gamma(u))$, it induces an ideal per-commit predicate $I_P(\omega(u))$. The desired commit condition is $I_P(\omega(u)) = 1$; multiset containment $\mu(u) \preceq B$ is one concrete instance. This paper addresses the representation obligation that the monitor's view must preserve every distinction needed to evaluate that predicate. The authenticated user, harness, or an external policy component supplies P; the planner may propose values but cannot create or enlarge the authority against which those values are checked.

Our runtime prototype instantiates one bounded policy, *provenance-origin confinement*. An untrusted observation may provide data, but it may not be the sole origin of a registered effect or effect-bearing value absent from the authenticated task. Thus the evaluated monitor tests whether a proposed call crosses this source boundary; it does not establish that the user is globally entitled to the action. ACLs, delegation, quotas, and organization policy remain independent authority predicates that could consume the same effect representation.

Trust assumptions and scope. Our conditional security argument requires a descriptor that covers in-scope effects, unforgeable provenance, sound task grounding, and a correct authority predicate. Every effectful call must be mediated, and the checked call and policy snapshot must govern the commit. Registration additionally assumes that the sandbox oracle observes the effects exercised by its interventions and that runtime invokes the registered implementation. If the planner can alter P, relabel provenance, or replace the checked call,

the stated confinement result does not hold. We do not address compromise of the host, monitor, provenance adapter, or tool implementation; post-registration code changes; effects outside the observed state boundary; output-only attacks; or dangerous actions explicitly requested by the authenticated user.

4 Counterfactual Effect Binding

Figure 2 separates two trust domains. Offline registration constructs and tests a tool descriptor against the registered implementation. Online mediation is deterministic: it instantiates the frozen descriptor for a concrete call and supplies the resulting effect view to a policy. The LLM may seed candidate names and bindings during registration, but neither its proposal nor a runtime agent decision is treated as authority.

We use three terms consistently. An *effect atom* is the policy-relevant unit in our model. A *tool descriptor* is the frozen output of offline registration. The *provenance-origin monitor* is our AgentDojo runtime instance: it expands descriptor fields into value records and rejects effects introduced only by marked untrusted evidence. A separate ToolSandbox mechanism instantiates concrete typed atoms and checks them against an effect bound. ToolSandbox isolates the atom interface; AgentDojo evaluates a conservative runtime instance over complete trajectories.

The registration evidence has two stages. A committed-state difference is an *effect witness*: it shows that the intervention changes what the tool does. A pair becomes an *authorization-separating witness* only when an admissible policy decides the two resulting semantic views differently. The first justifies retaining information about an effect; the second justifies a distinction in an authorization-sufficient representation.

4.1 From Committed Effects to Policy Views

Let $u = (t, \bar{x}, s, h)$ contain a tool name, totalized arguments, pre-state, and typed provenance evidence, and let $\mu(u)$ be the multiset of committed state changes produced by executing u. As in Section 3, $\omega(u)$ augments this multiset with policy-relevant trusted execution metadata. An *effect atom* records one occurrence that an admissible policy may need to authorize independently. For example, adding a participant to an existing calendar event can produce

$$\langle \mathsf{invite}, \mathsf{event{:}17}, \mathsf{sarah@example.com}, \mathsf{commit}, \mathsf{tool\text{-}output} \rangle.$$

Here the components denote effect kind, operated resource, target principal, operation or commit mode, and control provenance. A tool-specific atom may also retain a payload, visibility, date, or other qualifier when the policy family can distinguish its values. Lists expand per target or resource when their members can be authorized independently.

A general registered descriptor d_t contains (i) argument-totalization rules, (ii) proposed effect kinds, (iii) effect-bearing schema fields and their bindings or roles, (iv) inactive values, and (v) counterfactual evidence for those choices. Its instantiation function defines the monitor view

$$\rho_d(u) = A_{d_t}(u) = \text{instantiate}(d_t, \bar{x}, s, h),$$

a canonical multiset of concrete atoms. Thus $\mu(u)$ is what execution actually commits, whereas $\rho_d(u)$ is the effect representation available before commitment. The ToolSandbox mechanism implements this typed form for five source-executed tools. The AgentDojo prototype implements the narrower projection described above. Its field roles are audit annotations rather than independently validated semantic types: a field record is not automatically a complete resource–principal–operation atom.

Atom equality is field-wise after descriptor-defined canonicalization, and multiplicity is preserved: inviting the same principal to two resources is not collapsed into one occurrence. A general descriptor may bind aliases to stable resource or principal identifiers. The evaluated monitor has no such general resolver. Its frozen canonicalization handles bounded literal equivalences, including case and whitespace, numeric and date forms, and normalized URL forms. A typed resolver may admit a value only from a matching authorized-read ledger entry; it does not infer arbitrary entity aliases. We therefore evaluate alias-sensitive and typed-resource representations separately rather than assuming canonicalization is semantically complete.

Policy-relative minimality. Minimality is relative to a frozen intervention domain D and policy family $\mathcal{P}$. A representation is minimal on $(D, \mathcal{P})$ when merging any two of its classes creates a pair that some $P \in \mathcal{P}$ decides differently. Accordingly, changing committed state establishes that a field is effect-relevant. A trusted provenance or control-source change can also be authorization-relevant without changing state. Either establishes authorization necessity only when an admissible policy separates the resulting semantic views. A returned-string change alone establishes neither. This definition does not assert a unique or globally minimal tuple schema.

4.2 Offline Registration

Registration consumes a versioned tool schema and implementation, sandbox states, an intervention catalog, and, when authorization sufficiency is being tested, a policy oracle from $\mathcal{P}$. It proceeds as follows.

1. Propose. A developer or an LLM proposes effect names and field bindings from the tool name, description, and

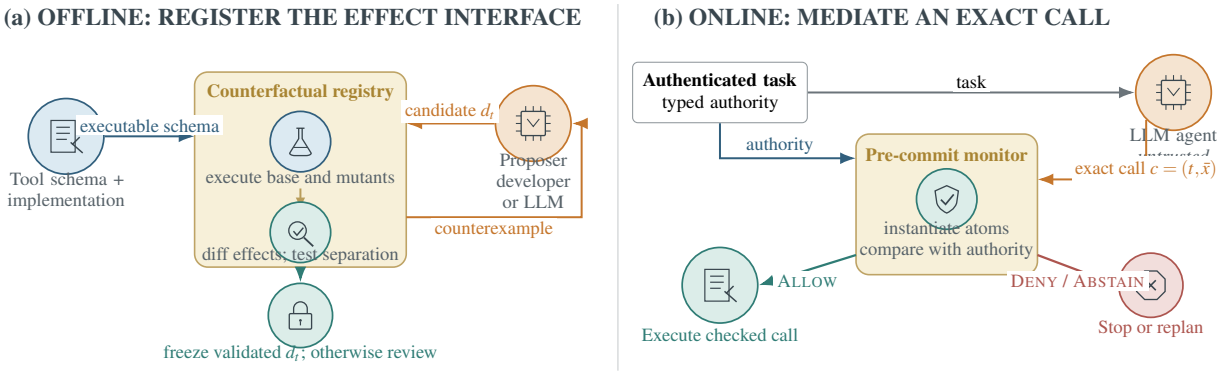

Figure 2: System overview. Offline (left), a proposer supplies a candidate effect descriptor, while counterfactual executions and policy-separating tests determine whether it is refined, frozen, or sent for review. Online (right), the deterministic monitor instantiates the frozen descriptor for the exact agent-proposed call, compares its effects with independently supplied authority, and commits only an Allow call.

schema. The origin of the candidate is immaterial to the security argument: it is an untrusted initial hypothesis and cannot determine registration. Our harness supports local-LLM seeding, but this paper does not evaluate automatic contract synthesis or onboarding-time reduction.

2. Intervene and execute. For a valid base call, the harness changes a field value or its presence while holding the implementation and initial state fixed. Interventions include typed alternatives, observed alternatives, boundary values, list expansion, and omission/default cases when supported. Surface-only variants serve as placebos. Both calls execute in copied sandboxes, so no external side effect is produced.

3. Classify evidence. A state-difference oracle compares committed state, returned output, and execution validity; provenance interventions additionally use the trusted harness annotation. The harness records *committed-effect changed*, *policy-metadata changed*, *output-only changed*, *semantic-invariant*, or *invalid/unresolved*. The first is evidence that a field participates in the exercised effect relation; the second records a change such as trusted control provenance. Either becomes an authorization-separating counterexample only if the selected policy oracle distinguishes the two semantic views. An invariant outcome supports omission only over the tested states and interventions; an invalid outcome is never treated as evidence of irrelevance.

4. Refine or freeze. A counterexample may add a field, split a list-valued effect, or introduce a qualifier into the candidate representation. Repeating this step implements the finite refinement procedure analyzed in Section 5. This is the general counterexample-guided loop; a descriptor is called authorization-sufficient only after its test domain contains no policy-separating collision. The evaluated AgentDojo registry uses a more conservative, nonminimal instance of this rule: it retains fields with effect-change evidence and fields whose tests remain unresolved. A later multi-base audit broadens the evidence but does not retroactively change that frozen registry. Thus the registry demonstrates coverage of exercised fields, not automatic discovery of a sparse minimum. The descriptor, implementation identity, intervention manifest, and evidence are hash-frozen together; runtime code cannot invoke the proposer.

Running example. Consider a calendar tool whose participant argument is a list. The proposal may initially describe the call only as create-event. Registration executes a base call that invites Alice and a paired call that additionally invites Eve. If the state oracle observes a second invitation and the policy allows Alice but not Eve, the pair is an authorization-separating counterexample. Refinement therefore expands the participant list into one invite atom per principal. A surface paraphrase that preserves the same event and participants should leave the atom multiset unchanged. The descriptor is frozen only for the intervention and policy domain exercised by these checks; neither test proves correctness for arbitrary calendars or future schema versions.

4.3 Runtime Interface

For context $u = (t, \bar{x}, s, h)$, the runtime first applies the descriptor's totalization function. In the AgentDojo monitor, a versioned schema catalog distinguishes required fields from explicit static defaults; inactive values such as an omitted optional update remain represented but generate no active effect check. A runtime-derived value is accepted only when a registered resolver relation and a matching typed authorized-read ledger entry justify it. Missing required values, unknown defaults, or unproved resolver values produce ABSTAIN or a

noncommitting replan decision. The runtime then instantiates $\rho_d(u)$ as defined above. In the general architecture, a policy interface

$$\text{Authorize}(\rho_d(u), P) \to \{\text{ALLOW}, \text{DENY}, \text{ABSTAIN}\}$$

receives these atoms and independently supplied authority context P. Such a policy may check ACLs, delegation, visibility, quotas, or other resource-specific constraints. Registration supplies the effect view; it does not manufacture this authority.

The authority source is explicit in each evaluated setting. Finite representation validation enumerates bounds over observed source-effect occurrences. The concrete ToolSandbox mechanism treats one frozen source-executed effect multiset as the bound and checks another call's typed atoms for multiset inclusion. AgentDojo supplies no general ACL: its bound is the narrower provenance-origin relation between the authenticated task, registered effects, and typed untrusted evidence. These settings test the representation and mediation interfaces under stated policies; they are not interchangeable evidence for deployment authority.

4.4 Provenance-Origin Monitor

The provenance-origin monitor instantiates a narrower policy than the general interface above. It asks whether explicitly untrusted observations introduced a registered effect or effect-bearing value absent from the authenticated task.

Structured provenance. The adapter traverses mappings, sequences, dataclasses, and Pydantic-like records before extracting explicitly marked untrusted control segments. This preserves provenance through structured tool outputs. Provenance identifies where a value came from; it does not by itself authorize the resulting effect.

Registered-field binding. For each registered field, the monitor expands mappings and lists to concrete leaf values. A value is rejected when it occurs in untrusted control text but is not literally grounded in the authenticated task. Per-value expansion prevents one task-grounded recipient from authorizing an additional injected recipient. The audit record stores the field, concrete value, descriptor role, provenance status, and decision reason.

Effect-label grounding. Some injected requests change the intended action without exposing a unique argument value. The monitor therefore performs a second, lexical check over action and object terms derived from the frozen tool name and effect label. If the untrusted segment requests that registered effect while the authenticated task does not, the call is denied. This is a benchmark-specific grounding heuristic, not general semantic authorization or concrete atom recovery.

Missing registration or failed totalization produces ABSTAIN; a detected untrusted expansion produces DENY; otherwise the monitor returns ALLOW. In the calendar example, an Eve value copied from explicitly untrusted text is denied when Eve is absent from the authenticated task; the monitor does not claim to infer whether the user has organization-level permission to invite her. The executor commits only the exact totalized call signature associated with ALLOW, and the audit log reconciles checked and executed calls. The monitor makes no runtime LLM call. ACLs and other authority predicates remain external to this evaluated policy instance.

5 Security Analysis

5.1 Why Representation Granularity Matters

For execution context u, let $\mu(u)$ be its committed effects and let $\gamma(u)$ contain policy-relevant metadata such as effect provenance. The ideal commit view is $\omega(u) = (\mu(u), \gamma(u))$. Each policy context $P \in \mathcal{P}$ induces an ideal decision $I_P(\omega(u))$. For example, our finite effect-set experiments use $I_B(\omega(u)) = [\mu(u) \preceq B]$; an ACL, delegation state, or quota snapshot could parameterize another predicate. The policy snapshot used for the check must also govern the commit. A monitor sees $\rho(u)$ rather than $\omega(u)$; a descriptor instantiates $\rho_d(u) = A_{d_t}(u)$.

Definition 1 (Authorization sufficiency)**.** A representation ρ is authorization-sufficient on domain D for policy family $\mathcal{P}$ when

$$\rho(u) = \rho(v) \Longrightarrow I_P(\omega(u)) = I_P(\omega(v))$$

for every $u, v \in D$ and $P \in \mathcal{P}$.

This definition is policy-relative. It does not require separating changes that no admissible policy distinguishes, and it does not privilege a fixed tuple of atom fields.

Theorem 2 (Representation collision)**.** *If ρ is not authorization-sufficient, no deterministic monitor whose only call-dependent input is $\rho(u)$ can, even when given P, both allow every authorized context and reject every unauthorized context.*

Proof sketch. Insufficiency gives u, v, P with equal representations and opposite ideal decisions. The monitor must return the same decision for both. Allowing admits the unauthorized member; denying or abstaining withholds the authorized member.

For any fixed P, the number of authorized rows in a representation cell that also contains an unauthorized row is therefore a lower bound on false denial or abstention for every sound monitor over that view. Tool-name, whole-call, and atom views are all subject to this result; an atom view is useful only when it preserves the distinctions required by the policy family.

5.2 Counterfactual Validation

An intervention $v = do_J(u)$ changes selected call fields, trusted provenance, or pre-state while holding the tool implementation fixed. Source execution observes committed state changes; the trusted harness records policy-relevant provenance and control metadata. Together they instantiate ω on the tested pair.

Proposition 3 (Counterfactual witness)**.** *If $\rho(u) = \rho(v)$ but some $P \in \mathcal{P}$ satisfies $I_P(\omega(u)) \neq I_P(\omega(v))$, then ρ is insufficient on every domain containing u and v.*

The proposition makes counterfactual testing a falsification procedure. An output-only change with unchanged ω is not a witness. The same committed state reached under a different trusted provenance may be a witness when an admissible policy distinguishes that provenance. Conversely, a representation change under a true semantic-invariant placebo indicates unnecessary sensitivity.

For a finite domain, counterexample-guided refinement repeatedly splits a representation cell using a witnessed separating field or qualifier. Call a cell *policy-mixed* when it contains u, v for which at least one $P \in \mathcal{P}$ gives different ideal decisions.

Theorem 4 (Finite refinement termination)**.** *On finite D, a refinement procedure that never merges cells can perform at most $|D| - 1$ nonempty cell splits. If each step separates a witnessed authorization collision and the procedure stops only when no policy-mixed cell remains, the resulting representation is authorization-sufficient on $(D, \mathcal{P})$.*

Proof sketch. Every split strictly increases the number of nonempty cells, which is bounded by $|D|$. At termination, equal representations imply membership in the same non-mixed cell. By the definition of policy-mixed, their ideal decisions agree for every $P \in \mathcal{P}$.

Generated interventions need not cover future calls. We therefore retain unresolved interventions instead of treating them as evidence of irrelevance.

5.3 Conditional Atom-Level Mediation

The representation results become a commit guarantee only when the descriptor, policy, and enforcement path satisfy separate obligations. Let J_P be the ideal policy predicate expressed over a descriptor's atom view.

Theorem 5 (Conditional atom-level mediation)**.** *Assume four obligations for every reachable effectful call u. First, descriptor fidelity gives $J_P(A_{d_t}(u)) = I_P(\omega(u))$ for every admissible P. Second, the authorizer returns* ALLOW *only when $J_P(A_{d_t}(u)) = 1$. Third, every effectful call is mediated before commit. Finally, the executor commits exactly the checked call under the same policy snapshot. Then every allowed committed call satisfies $I_P(\omega(u)) = 1$.*

Proof sketch. Consider an allowed committed call u. Complete mediation supplies a check for u, and authorizer soundness gives $J_P(A_{d_t}(u)) = 1$. Descriptor fidelity therefore gives $I_P(\omega(u)) = 1$. Call and policy-snapshot check–use consistency make this conclusion apply at commitment.

The theorem separates representation fidelity from policy correctness. Counterfactual tests probe the first obligation; ToolSandbox instantiates the atom interface on a finite domain. The AgentDojo monitor below enforces a narrower provenance-origin predicate over registered field values.

5.4 Implemented Provenance-Origin Confinement

For a registered call u, let $F(u)$ be its security-field values and let $E(u)$ be its set of registered effect kinds. Let $U_V(h)$ and $U_E(h)$ be values and effect requests extracted from provenance-marked untrusted evidence h. Let $G_V(q)$ and $G_E(q)$ be values and effects grounded in the authenticated task. For this monitor, "introduced solely" means membership in the relevant untrusted set without membership in the corresponding grounded set; it is not a claim of general causal attribution. The implemented provenance-origin monitor allows only when

$$F(u) \cap U_V(h) \subseteq G_V(q) \quad \text{and} \quad E(u) \cap U_E(h) \subseteq G_E(q).$$

Corollary 6 (Conditional provenance-origin confinement)**.** *Assume that the descriptor covers every effect-bearing field and effect kind governed by the monitor. The provenance adapter identifies every untrusted value and effect request influencing the call, and grounding marks only values and effects supported by the authenticated task. Every effectful call is mediated, and the executor commits exactly the checked call. Then an allowed call contains no registered effect or effect-bearing value introduced solely by untrusted evidence outside the authenticated task.*

Proof sketch. Suppose an allowed call contains such an effect-bearing value. Descriptor, provenance, and grounding soundness place it in $F(u) \cap U_V(h)$ but not $G_V(q)$, contradicting the first allow condition. If only an effect request changes, the same obligations place it in $E(u) \cap U_E(h)$ but not $G_E(q)$, contradicting the second condition. Mediation and exact execution transfer the decision to the committed call.

This corollary covers the implemented policy, not ACL authorization, output-only goals, or dangerous authenticated requests. Runtime audits test mediation and check–use consistency; source interventions test descriptor adequacy on the exercised domain. Appendix D gives the complete definitions and proof details.

6 Evaluation Methodology

We organize the evaluation around four research questions that follow the paper's evidence chain. RQ1 asks whether tool calls actually commit compound or pre-state-dependent effects. RQ2 asks whether coarse representations lose policy-relevant distinctions and whether counterfactual registration recovers them. RQ3 asks whether concrete atoms can implement a frozen authorization relation. RQ4 asks what security, utility, and audit properties a descriptor-driven runtime monitor provides. The questions use separate oracles and are not interchangeable: runtime attack success, for example, cannot establish representation sufficiency.

RQ1: Do calls commit independently authorizable effects? We replay all 339 calls in AgentDojo v1.1.2's 97 official benign trajectories from fresh prescribed states. Before/after differences are normalized into resource, target, visibility, message, membership, and external-interaction effects. We count both the number of logical effect occurrences and whether a call crosses effect types or benchmark subsystems.

For a controlled representation test, we use 56 calls to five source-verified AgentDojo tools. Under the power set of observed effect occurrences, unequal concrete effect multisets are authorization-separable. Grouping calls by tool name, common fields, raw arguments, or typed effects therefore exposes exactly which policy distinctions each representation erases.

RQ2: Can counterfactual registration recover required distinctions? For each registered AgentDojo field, the harness attempts observed-value, typed, boundary, list-expansion, omission/default, and surface-only variants. It records committed-state change, output-only change, invariance, invalidity, and unresolved outcomes separately. A state-changing pair establishes effect relevance over the exercised intervention family. It establishes authorization necessity only when the policy oracle also separates the pair. The initial intervention suite is used to refine the candidate representation, not to estimate held-out generalization.

We therefore freeze a second protocol over five previously unused ToolSandbox tools and 32 call/state contexts [19]. These contexts exercise values, preconditions, defaults, and compound effects; invalid contexts remain in the results. We compare the source-effect partition with partitions induced by common fields and frozen typed atoms. The provenance component used later at runtime comes from trusted benchmark evidence boundaries and is evaluated as a policy input rather than source-validated as a tool semantic.

RQ3: Can atoms realize the frozen policy relation? The 32 ToolSandbox contexts also define a finite mechanism test. Within each tool, every context serves as an authority anchor and is paired with every candidate context, producing 232 ordered queries. The source oracle allows a candidate exactly when its executed effect multiset is a submultiset of the anchor's. We compare tool identity, canonical exact arguments, common effect fields, concrete typed atoms, and the source oracle. This isolates whether the atom interface can implement the specified relation; the anchors are finite source-derived bounds, not deployment ACLs.

RQ4: What does descriptor-driven runtime mediation establish? AgentDojo supplies 97 benign tasks and 629 official `important_instructions` attack pairs across workspace, Slack, travel, and banking. Under DeepSeek, we compare no guard, Spotlighting, and Provenance Monitor with the same model, decoding, benchmark version, task keys, and validators. Benign utility is repeated four times per method in interleaved order. A second common-protocol comparison runs no guard, Spotlighting, Prompt Sandwiching, a PromptArmor-style local adapter, and Provenance Monitor on all 726 keys under one local Qwen3-32B checkpoint. These adapters are comparable implementations, not original-paper reproductions.

Generalization tests use a frozen 320-case manifest spanning four suites and four public attack templates, plus a separate 40-case bounded-search diagnostic. Neither permits result-informed repair. For cross-environment transfer, we replay 303 fixed public AgentLAB attack artifacts under AgentDojo v1.2.1 [17]. Matched no-guard and provenance-normalized Provenance Monitor conditions use the same Qwen3-32B checkpoint and case manifest. This transfer tests saved evidence under a new harness; it does not reproduce AgentLAB's cumulative attack optimization.

We separate runtime security from mechanism attribution. A retrospective diagnostic applies four views to the same completed no-guard trajectories and measures only whether a view could intercept a call. A closed-loop 321-case applicability subset instead reruns no blocking, whole-call provenance, effect-only checking, Provenance Monitor, and a raw-field taint control. The control holds task grounding, provenance, totalization, and execution fixed while removing descriptor effect-label expansion. This comparison tests whether registered effect semantics change decisions; source-executed collisions remain the independent evidence for representation quality because the AgentDojo registry retains every schema field.

Metrics and integrity. Official attack success and task utility use AgentDojo's native validators; no LLM judges either outcome. We also report coverage, abstention, direct denials, runtime LLM calls, and executions without a preceding allow. Exact-key security comparisons use McNemar tests. Benign utility uses task-clustered bootstrap intervals, with non-inferiority requiring the one-sided 95% lower bound to exceed -0.05. The finite authorizer reports unsafe pre-allow,

Table 1: Source-executed effects in AgentDojo v1.1.2 benign trajectories ($N = 339$ calls). Rates after the first row use the 100 effectful calls as the denominator.

Measure	Rate
Effectful calls	29.5%
Compound effects	17.0%
Heterogeneous effects	13.0%
Multiple targets	7.0%
Cross-subsystem effects	13.0%

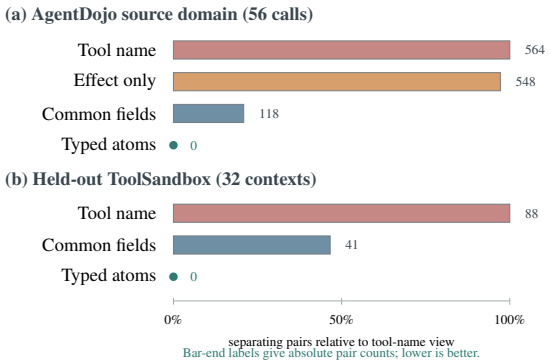

Figure 3: Representation collision falls as the authorization view preserves execution-relevant effects. Bars normalize policy-separating pair counts to the tool-name view within each domain; labels report absolute counts. Typed atoms reach the source-effect partition in both the development and held-out domains.

false denial, coverage, abstention, and accuracy over all 232 queries. Every protocol freezes its model, descriptor, case manifest, and monitor identity; failed or missing rows remain visible, and no call reaches an external service. Appendix B gives the fail-fast reproduction checks.

7 Results

7.1 RQ1: Do Calls Commit Compound Effects?

Compound effects occur in a widely used agent benchmark. Source replay finds a state or external-interaction effect in 29.5% of 339 AgentDojo calls. Among the effectful calls, 17.0% contain multiple logical effect units and 13.0% cross effect types or benchmark subsystems (Table 1).

7.2 RQ2: Can Registration Recover Required Distinctions?

Coarse-view collisions. Coarse views lose distinctions that a policy can require. In the 56-call source-executed domain, tool name and effect-only views create 564 and 548 authorization-separating pairs. Common fields reduce this to 118 but still merge payload, temporal, recurrence, and repeated-effect distinctions. The typed effect representation matches the source-effect partition with no mixed cell (Figure 3(a)). This establishes sufficiency for the finite call domain and submultiset policy family.

Table 2: Multi-base source-executed registration audit over 75 suite-scoped field instances and 1,325 intervention executions. "Witness" means that a valid mutation changed committed state.

Registration outcome	Rate
Unique fields retained	100.0%
Fields with ≥ 5 valid intervention kinds	50.7%
Fields with a committed-effect witness	88.0%
Valid intervention executions	76.8%
Committed-effect-changing executions	73.3%
Output-only executions	0.8%
Effect-invariant executions	2.7%
Invalid or unresolved executions	23.2%

A frozen refinement lattice connects this endpoint to the procedure. Adding date, subject, recurrence, payload, and visibility qualifiers in sequence reduces separating pairs from 124 to 60, 28, 12, 4, and zero. All 211 comparable edges in the 32-representation lattice preserve partition refinement, and none increases the separating-pair or ambiguous-cell lower bound. The lattice's 124-pair erased endpoint applies five role-specific deletion and masking operations; it is intentionally coarser than the 118-pair common-field view in Figure 3(a), which only removes the qualifier map. Both reach the same typed endpoint. This is an empirical instance of Theorem 4; other domains may require a different qualifier order.

Counterfactual repair and held-out transfer. The initial source-grounded interventions ($N = 80$) falsify the common-field representation in 18.8% of cases. Adding typed payload, time, recurrence, and setting qualifiers removes those observed gaps, but this is a post-failure repair on the same suite. We therefore evaluate the frozen representation on a separate source.

The pre-registered ToolSandbox domain exhibits the same gap. Across 32 contexts, common fields retain four mixed cells and 41 separating pairs. Typed effects reproduce the 25-cell source partition without collision or overpartition. In 28.1% of contexts, a witness keeps the tool and explicit arguments fixed while pre-state changes the committed effect, showing why argument values alone are not a complete effect representation (Figure 3(b)).

AgentDojo-wide registration is more conservative. The runtime descriptor retains 100.0% of 67 unique fields. Searching multiple valid base calls and adding omission/default tests yields committed-effect witnesses for 88.0% of 75 suite-

Table 3: Finite ToolSandbox authorizer check. UPA is unsafe pre-allow; FD is false denial.

View	UPA	FD	Coverage	Accuracy
Tool name	100.0%	0.0%	100.0%	35.3%
Exact raw args	9.3%	34.1%	100.0%	81.9%
Common fields	56.7%	0.0%	100.0%	63.4%
Concrete atoms	0.0%	0.0%	100.0%	100.0%
Source oracle	0.0%	0.0%	100.0%	100.0%

All 232 ordered queries use one executed context as the allowed effect multiset and check another context of the same tool.

scoped field instances, but only 50.7% have at least five valid intervention kinds. Invalid or unresolved executions make up 23.2% of 1,325 attempts, mainly because mutations use nonexistent resources or violate preconditions. The audit supports effect relevance for exercised fields, not sparse atom discovery (Table 2).

7.3 RQ3: Can Atoms Realize the Frozen Policy?

Concrete atoms exactly realize the finite ToolSandbox decision relation. In all 232 ordered context pairs, a candidate is authorized exactly when its source-executed effect multiset is contained in the authority anchor's multiset. Tool identity has 100.0% UPA. Exact raw arguments reduce UPA to 9.3% but falsely deny 34.1% of authorized pairs; common fields have 56.7% UPA. Concrete typed atoms match the source oracle with 0.0% UPA and 0.0% false denial (Table 3). The anchors and ideal decisions are source-derived effect bounds, not deployment ACLs.

7.4 RQ4: What Does Runtime Mediation Establish?

Official AgentDojo evaluation. On all 629 official DeepSeek attack pairs, attack success is 1.0% without a guard and 0.0% with the provenance-origin monitor. Excluding the case used to repair structured provenance, and excluding 20 output-only goals outside the method's scope, leaves 1.0% versus 0.0% over 608 pairs (one-sided exact McNemar $p = .0156$; Figure 4(a)). The 1.0-percentage-point estimate comes from a low baseline attack rate.

The reduction is not produced by abstaining on most calls. Across 2,783 pre-commit checks, the monitor allows 96.8%, denies 3.2%, abstains on none, makes no runtime guard LLM call, and records no execution without ALLOW. No benign call is directly denied in the provenance-origin sanity run. Nevertheless, attack-task utility is 72.0%, versus 76.3% without a guard and 75.8% under Spotlighting. On paired tasks, 13.7% succeed only without the guard and 9.4% only with it, so the net loss is not identical to the direct block count.

Spotlighting also reaches 0.0% attack success and has higher utility in this run. The result shows that a registered descriptor can drive deterministic pre-commit decisions with an exact audit trail, but it does not establish numerical dominance or an atom-specific security gain. Repeated benign utility and held-out attack families use separately frozen protocols below; the fresh second-checkpoint comparison remains pending.

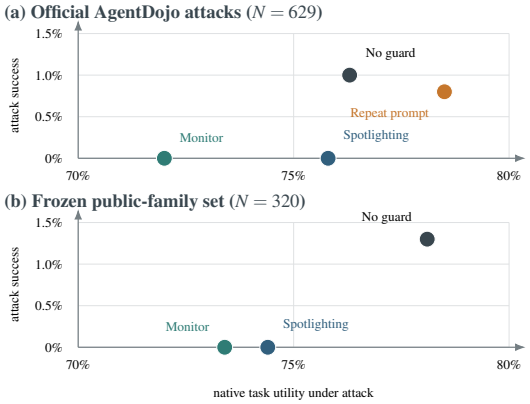

Figure 4: Security–utility tradeoff under one DeepSeek model. Lower attack success and higher task utility are preferable. The monitor removes the observed injection successes in both protocols, but does not dominate the prompting baseline on utility. Panels use separate frozen case manifests and must be interpreted within, not across, protocols.

Table 4: Closed-loop monitor-view comparison and fixed saved-attack transfer. Panel (a) uses 48 benign and 273 attack cases from a selection-conditioned AgentDojo subset. Panel (b) replays 303 fixed AgentLAB task-injection artifacts saved under AgentDojo v1.2.1, while the main AgentDojo results use v1.1.2; it is not adaptive attack generation.

Setting	Benign utility	Attack utility	ASR
(a) Monitor representation, AgentDojo v1.1.2			
No blocking	62.5%	56.0%	13.9%
Whole-call provenance	81.3%	21.2%	4.0%
Effect only	77.1%	53.8%	7.7%
Raw-field taint	81.3%	59.3%	3.7%
Registered fields	64.6%	54.2%	2.6%
(b) Fixed AgentLAB transfer, AgentDojo v1.2.1			
No defense	–	61.1%	31.4%
Provenance-origin monitor	–	43.6%	0.3%

Attribution and transfer. On the 321-case applicability subset, the provenance-origin monitor records 2.6% attack success. Whole-call provenance, effect-only checking, raw-field taint, and no blocking record 4.0%, 7.7%, 3.7%, and 13.9%, respectively. The monitor's benign utility is 64.6%, versus 81.3% for whole-call and raw-field checking (Table 4(a)). Because field-level applicability selected this subset, these are mechanism comparisons rather than benchmark-wide estimates.

The retrospective four-view diagnostic explains the selec-

tivity tradeoff on the completed no-guard trajectories. Whole-call provenance blocking would intercept 98.1% of successful attacks but touch 53.1% of 629 attack trajectories. Effect-only checking intercepts 41.5% and touches 7.5%; the provenance-origin monitor intercepts 92.5% and touches 14.9%. No view touches the 97 injection-free trajectories because they contain no marked untrusted control segment. These are fixed-trajectory interception counts, not counterfactual ASR or utility: a live denial can change all later actions.

The fixed AgentLAB transfer provides bounded evidence for multi-step mediation. Under one Qwen3-32B checkpoint and the same 303-case manifest, attack success falls from 31.4% to 0.3%, while attack-task utility falls from 61.1% to 43.6%. All executed guarded calls have an exactly matching pre-commit record (100.0%, $N = 1{,}334$). This is a substantial security transfer with a 17.5-percentage-point utility cost in this saved replay.

The completed raw-field control sharpens the attribution boundary. On the same subset, raw-field taint records 3.7% attack success and 81.3% benign utility, compared with 2.6% and 64.6% for the provenance-origin monitor. The two views disagree on 3.6% of 788 effectful checks, always when raw-field taint allows and the monitor denies. Effect semantics therefore change decisions, but do not improve selectivity or utility on this subset.

Repeated utility and held-out attacks. Across four interleaved DeepSeek repetitions, benign utility is 77.6% without a guard, 78.6% under Spotlighting, and 76.5% under Provenance Monitor. The mean monitor-minus-no-guard difference is -1.0 percentage point. The task-clustered one-sided 95% lower bound is -5.4 percentage points, below the pre-registered -5.0-point margin, so benign-utility non-inferiority is not established.

On the frozen 320-case public-family set, attack success is 1.3% without a guard and 0.0% under both Spotlighting and Provenance Monitor. Native task utility on the same rows is 78.1%, 74.4%, and 73.4%, respectively. This held-out result retains the observed security reduction but also confirms that the runtime profile is not utility-neutral or numerically dominant over the prompting baseline (Figure 4(b)).

Together, RQ1–RQ3 establish the finite-domain representation and mechanism claims. RQ4 establishes audited descriptor-driven mediation, while its raw-field and prompting controls delimit the current runtime contribution. Appendix C reports registration failures, sensitivity, controlled monitor stresses, and cost.

8 Limitations

Registration coverage. Counterfactual registration tests only exercised semantics. The AgentDojo registry retains all 67 schema fields, and only 50.7% of 75 suite-scoped field instances obtain five valid intervention kinds. Field roles are lexical heuristics. Our minimality claim is therefore relative to a checked domain and policy family; a state-changing witness establishes effect relevance, while authorization necessity also requires a policy-separating pair.

Trusted interfaces. The runtime assumes complete provenance for mediated effects. Its adapter recognizes explicit untrusted-content boundaries, but literal canonicalization can confuse values such as `10` and `10,000`, and structured traversal does not cover arbitrary harnesses or semantic paraphrases. The implemented policy detects effect-bearing values introduced by marked untrusted observations. ACLs, delegation, quotas, authenticated dangerous requests, and output-only goals require other policy machinery. The planner cannot be the sole source of that policy: if the same compromised model can enlarge its authority context, atomization becomes structured self-approval rather than a security boundary.

Safety and utility. The provenance-origin monitor and raw-field control disagree on 3.6% of 788 effectful checks. Effect semantics therefore change enforcement, but the monitor does not improve selectivity on that subset. DeepSeek's no-guard attack rate is only 1.0%, Spotlighting reaches the same observed 0.0% attack rate as our monitor, and benign non-inferiority is not established. The fixed AgentLAB transfer reduces attack success but costs 17.5 percentage points of utility. We report second-model, repeated-utility, locked-family, and transfer experiments separately because their populations and protocols differ.

9 Conclusion

Tool-effect binding is the representation obligation that precedes runtime authorization. Source execution establishes what a tool commits; a policy witness establishes which effects require different decisions; and a representation collision shows whether the monitor preserves that distinction. When it does not, downstream enforcement cannot recover the lost information. Source-executed counterfactuals expose these collisions and guide descriptor refinement before registration.

Across finite AgentDojo and pre-registered ToolSandbox domains, typed effects remove collisions left by common call views; on ToolSandbox, concrete atoms exactly realize the specified finite policy relation. In AgentDojo, a provenance-origin monitor uses the registered descriptor for deterministic pre-commit checks and complete call reconciliation. Its security gains remain subject to provenance quality and a visible utility tradeoff. The contribution is an executable and falsifiable effect representation, not an authority source: the planner proposes calls, while an independent policy must decide which represented effects may commit.

References

[1] Hengyu An, Jinghuai Zhang, Tianyu Du, Chunyi Zhou, Qingming Li, Tao Lin, and Shouling Ji. Ipiguard: A novel tool dependency graph-based defense against indirect prompt injection in llm agents, 2025. EMNLP 2025.

[2] Rahul Suresh Babu and Laxmipriya Ganesh Iyer. Contract2Tool: Learning preconditions and effects for reliable tool-augmented LLM agents, 2026.

[3] Sander Beckers and Joseph Y. Halpern. Abstracting causal models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 33, pages 2678–2685, 2019.

[4] Peter Buneman, Sanjeev Khanna, and Wang-Chiew Tan. Why and where: A characterization of data provenance. In *Database Theory – ICDT 2001*, volume 1973 of *Lecture Notes in Computer Science*, pages 316–330. Springer, 2001.

[5] James Cheney, Laura Chiticariu, and Wang-Chiew Tan. Provenance in databases: Why, how, and where. *Foundations and Trends in Databases*, 1(4):379–474, 2009.

[6] Edmund M. Clarke, Orna Grumberg, Somesh Jha, Yuan Lu, and Helmut Veith. Counterexample-guided abstraction refinement. In *Computer Aided Verification*, volume 1855 of *Lecture Notes in Computer Science*, pages 154–169. Springer, 2000.

[7] Edoardo Debenedetti, Ilia Shumailov, Tianqi Fan, Jamie Hayes, Nicholas Carlini, Daniel Fabian, Christoph Kern, Chongyang Shi, Andreas Terzis, and Florian Tramer. Defeating prompt injections by design, 2025.

[8] Edoardo Debenedetti, Jie Zhang, Mislav Balunovic, Luca Beurer-Kellner, Marc Fischer, and Florian Tramer. Agentdojo: A dynamic environment to evaluate prompt injection attacks and defenses for llm agents, 2024.

[9] Dorothy E. Denning. A lattice model of secure information flow. *Communications of the ACM*, 19(5):236–243, 1976.

[10] Jack B. Dennis and Earl C. Van Horn. Programming semantics for multiprogrammed computations. *Communications of the ACM*, 9(3):143–155, 1966.

[11] Linfeng Fan, Ziwei Li, Yuan Tian, Yichen Wang, Rongsheng Li, and Xiong Wang. The granularity mismatch in agent security: Argument-level provenance solves enforcement and isolates the LLM reasoning bottleneck, 2026.

[12] Atticus Geiger, Zhengxuan Wu, Hanson Lu, Josh Rozner, Elisa Kreiss, Thomas Icard, Noah D. Goodman, and Christopher Potts. Inducing causal structure for interpretable neural networks. In *Proceedings of the 39th International Conference on Machine Learning*, volume 162 of *Proceedings of Machine Learning Research*, pages 7324–7338. PMLR, 2022.

[13] Norm Hardy. The confused deputy: (or why capabilities might have been invented). *ACM SIGOPS Operating Systems Review*, 22(4):36–38, 1988.

[14] Yu He, Haozhe Zhu, Yiming Li, Shuo Shao, Hongwei Yao, Zhihao Liu, and Zhan Qin. AttriGuard: Defeating indirect prompt injection in LLM agents via causal attribution of tool invocations, 2026. Accepted by USENIX Security 2026.

[15] Yue Huang, Hang Hua, Yujun Zhou, Pengcheng Jing, Manish Nagireddy, Inkit Padhi, Greta Dolcetti, Zhangchen Xu, Subhajit Chaudhury, Ambrish Rawat, Liubov Nedoshivina, Pin-Yu Chen, Prasanna Sattigeri, and Xiangliang Zhang. Building a foundational guardrail for general agentic systems via synthetic data, 2025.

[16] Laxmipriya Ganesh Iyer and Rahul Suresh Babu. The gate is only as honest as its contracts: ContractGuard for the contract layer of risk-aware causal gating, 2026.

[17] Tanqiu Jiang, Yuhui Wang, Jiacheng Liang, and Ting Wang. AgentLAB: Benchmarking LLM agents against long-horizon attacks, 2026.

[18] Minbeom Kim, Mihir Parmar, Phillip Wallis, Lesly Miculicich, Kyomin Jung, Krishnamurthy Dj Dvijotham, Long T. Le, and Tomas Pfister. CausalArmor: Efficient indirect prompt injection guardrails via causal attribution, 2026.

[19] Jiarui Lu, Thomas Holleis, Yizhe Zhang, Bernhard Aumayer, Feng Nan, Haoping Bai, Shuang Ma, Shen Ma, Mengyu Li, Guoli Yin, Zirui Wang, and Ruoming Pang. ToolSandbox: A stateful, conversational, interactive evaluation benchmark for LLM tool use capabilities. In *Findings of the Association for Computational Linguistics: NAACL 2025*, pages 1160–1183, Albuquerque, New Mexico, April 2025. Association for Computational Linguistics.

[20] John M. Lucassen and David K. Gifford. Polymorphic effect systems. In *Proceedings of the 15th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, pages 47–57. Association for Computing Machinery, 1988.

[21] Jiaqi Luo, Songyang Peng, Jiarun Dai, Zhile Chen, Zhuoxiang Shen, Geng Hong, Xudong Pan, Yuan Zhang,

and Min Yang. AgentGuard: An attribute-based access control framework for tool-use LLM-based agent, 2026.

[22] Yuhan Ma and Stefan Schmid. SecureClaw: Clawing back control of LLM agents, 2026.

[23] Yutao Mou, Zhangchi Xue, Lijun Li, Peiyang Liu, Shikun Zhang, Wei Ye, and Jing Shao. Toolsafe: Enhancing tool invocation safety of llm-based agents via proactive step-level guardrail and feedback, 2026. Work in progress.

[24] Judea Pearl. *Causality: Models, Reasoning, and Inference*. Cambridge University Press, 2 edition, 2009.

[25] Arun Ravindran and Saurabh Deochake. ToolGuardian: Declarative security for AI agent-tool interactions, 2026.

[26] Yangjun Ruan, Honghua Dong, Andrew Wang, Silviu Pitis, Yongchao Zhou, Jimmy Ba, Yann Dubois, Chris J. Maddison, and Tatsunori Hashimoto. Identifying the risks of lm agents with an lm-emulated sandbox, 2023.

[27] Jerome H. Saltzer and Michael D. Schroeder. The protection of information in computer systems. *Proceedings of the IEEE*, 63(9):1278–1308, 1975.

[28] Igor Santos-Grueiro. Temporary authority, permanent effects: Commit-time authorization for LLM agents, 2026.

[29] Tianneng Shi, Jingxuan He, Zhun Wang, Hongwei Li, Linyu Wu, Wenbo Guo, and Dawn Song. Progent: Securing ai agents with privilege control, 2025.

[30] Vincent Siu, Jingxuan He, Kyle Montgomery, Zhun Wang, Neil Gong, Chenguang Wang, and Dawn Song. A framework for formalizing LLM agent security, 2026.

[31] Huiri Tan, Yikun Wang, Puyang Zhang, Shangyu Li, and Jiasi Shen. ETAS: An effect-typed language for agent systems, 2026.

[32] Peiran Wang, Ying Li, and Yuan Tian. Aligning provenance with authorization: A dual-graph defense for LLM agents, 2026.

[33] Wei Zhao, Zhe Li, Peixin Zhang, and Jun Sun. ClawGuard: A runtime security framework for tool-augmented LLM agents against indirect prompt injection, 2026.

[34] Jinhao Zhu, Kevin Tseng, Gil Vernik, Xiao Huang, Shishir G. Patil, Vivian Fang, and Raluca Ada Popa. Miniscope: A least privilege framework for authorizing tool calling agents, 2025.

[35] David Mellafe Zuvic. Capability gates are not authorization: Confused-deputy failures in LLM agent frameworks, 2026.

Table 5: Representation collisions over 56 source-executed AgentDojo calls. "Pairs" counts authorization-separating pairs under the finite submultiset policy family.

Representation	Mixed	Pairs	Sufficient
Tool name	5	564	no
Effect only	6	548	no
Common fields	5	118	no
Reviewed common contract	5	118	no
Reviewed typed contract	0	0	yes
Full source effect	0	0	yes

A Ethical Considerations

All tool executions occur in public benchmark sandboxes or copied local state. No experiment sends a message, transfers money, changes a cloud resource, or commits another real external effect. API models receive benchmark task text and synthetic benchmark content, not private user data. Credentials are provided only through environment variables and are excluded from logs and artifacts. Released adversarial material is limited to public benchmark families and the fixed evaluation records needed to reproduce defensive claims. Incorrect descriptors or misplaced confidence could create harm; accordingly, the paper reports unresolved registration cases and negative utility results and makes no deployed-system guarantee.

B Open Science

The artifact package records source-executed effect oracles, frozen intervention and case manifests, conservative descriptors, row-level benchmark outputs, official-validator results, model identifiers and hashes, runtime decisions, tests, and environment locks. A fail-fast status manifest identifies the frozen artifacts required by each reported result; the release package includes only claims whose required artifacts and keys pass that check. Sidecar labels and source effects are used only for scoring and do not enter model prompts or deployable inputs. Before release, the package is rebuilt from the final manuscript state and scanned to exclude credentials, absolute paths, repository history, and identifying metadata.

Protocols hash model files, descriptors, case manifests, registered source, and monitor code. Finalizers reject duplicate or missing keys, execution errors, method-specific denominators, changed sources, and mismatches between pre-commit records and executed calls. Held-out cases use a frozen hash ordering recorded in the artifact.

C Additional Evaluation Detail

Exact representation diagnostics. Figure 3 visualizes the separating-pair lower bound. Tables 5 and 6 retain the exact

Table 6: Pre-registered held-out validation on 32 contexts for five public ToolSandbox tools. "Coll." counts representation cells containing distinct source-observed effect sets; "Sep." counts authorization-separating pairs. All eight expected invalid call/state contexts remain in the denominator.

Representation	Cells	Coll.	Sep.	Overpart.
Tool name	5	5	88	16
Common fields	11	4	41	0
Typed contract	25	0	0	0
Source effect	25	0	0	0

Table 7: Same-model DeepSeek AgentDojo attack comparison ($N = 629$). AU is utility under attack. The scope-aligned comparison uses $N = 608$ after excluding one disclosed repair case and 20 output-only goals.

Method	AU	Attack success
No guard	76.3%	1.0%
Repeat user prompt	78.5%	0.8%
Spotlighting	75.8%	0.0%
Provenance Monitor	72.0%	0.0%
Scope aligned: no guard 1.0%; Provenance Monitor 0.0% ($p = .0156$).		

cell, collision, and overpartition diagnostics.

Exact DeepSeek runtime results. Table 7 retains the exact same-model values and the scope-aligned comparison summarized by Figure 4.

Controlled monitor stresses. Released checkpoints respond to some effect, resource, and authorization changes, but none binds every tested axis jointly. The hard guard's UPA rises from 3.6% on the unified suite to 38.3% on the repaired resource/authorization stress. Removing resource or authorization matching raises UPA to 61.7% and 89.2%, respectively. These are adapted custom stresses rather than original-paper benchmark reproductions.

Registration failures. The multi-base AgentDojo audit attempts 1,325 source executions over 75 suite-scoped field instances. Of these, 76.8% are valid, 73.3% change committed state, 0.8% change only the returned output, 2.7% are effect-invariant, and 23.2% are invalid or unresolved. Most invalid interventions substitute nonexistent resource identifiers or violate a tool precondition. We retain those fields in the descriptor; absence of a valid witness is never interpreted as irrelevance. The artifact includes one row per attempted execution and a per-field summary.

Bounded public-family sensitivity. The current diagnostic fixes 40 official attack keys and crosses each with four public AgentDojo attack generators. Worst-of-four attack success

Table 8: Controlled counterfactual problem characterization. Panel (a) reports group-level sensitivity to effect, authorization, and resource changes; higher is better. Panel (b) reports row-level unsafe pre-allow (UPA), safe false denial (FD), and coverage. These adapted custom stresses are not original-paper benchmark reproductions.

(a) Partial binding across monitors

Method	Effect	Auth.	Resource	UPA
Tool-name proxy	0.0%	0.0%	0.0%	29.2%
Qwen self-audit	66.7%	56.5%	29.2%	0.4%
TS-Guard	62.5%	76.4%	83.3%	15.9%
Safiron	100.0%	8.8%	16.7%	54.5%

(b) Hard-guard granularity stress

Guard / stress	Rows	UPA	FD	Coverage
Unified suite	822	3.6%	6.5%	91.7%
Resource/auth.	240	38.3%	0.0%	92.1%
w/o resource	240	61.7%	0.0%	93.8%
w/o authorization	240	89.2%	0.0%	92.1%
Provenance	336	0.0%	18.8%	54.5%

is 10.0% without defense and 5.0% under the provenance-origin monitor; terminal utility is 45.0% and 42.5%. The four attack-discordant pairs give two-sided exact McNemar $p = .625$. This underpowered result is a bounded sensitivity check, not evidence of unrestricted adaptation or a statistically established security improvement.

Cost accounting. The deterministic comparison kernel has sub-millisecond median latency in the reported microbenchmarks. This excludes descriptor registration, model inference, sandbox execution, and I/O. Historical complete-trajectory runs have median duration 53.27 s without mediation and 85.54 s under an earlier mediation protocol, but they were sequential and used different recovery paths, so the difference is observational rather than a causal overhead estimate. The current provenance-origin guard makes no runtime LLM call; offline registration bears all proposal and counterfactual-execution cost.

D Formal Definitions and Proof Details

Decision model. Let D be a finite set of reachable execution contexts and $\mathcal{P}$ the admissible per-commit policy contexts. For $u \in D$, $\Delta(u)$ is the finite ordered trace of security-relevant concrete effect occurrences, $\mu(u)$ is its induced multiset, and $\gamma(u)$ is the policy-relevant execution metadata associated with the commit. The ideal semantic view is $\omega(u) = (\mu(u), \gamma(u))$. Each $P \in \mathcal{P}$ induces the ideal decision $I_P(\omega(u))$; multiset containment is the instance $I_B(\omega(u)) = [\mu(u) \preceq B]$. The snapshot P used by the check is also the snapshot governing the corresponding commit. A deterministic monitor over representation ρ is

$$M : \mathrm{range}(\rho) \times \mathcal{P} \to \{\textsc{Allow}, \textsc{Deny}, \textsc{Abstain}\}.$$

It is sound on D when $M(\rho(u), P) = \text{ALLOW} \Rightarrow I_P(\omega(u)) = 1$. It is permissive when $I_P(\omega(u)) = 1 \Rightarrow M(\rho(u), P) = \text{ALLOW}$. Abstention may preserve safety but does not commit authorized work, so it is a loss of permissiveness in this statement.

Proof of Theorem 2. Because ρ is not authorization-sufficient, there are $u, v \in D$ and $P \in \mathcal{P}$ with $\rho(u) = \rho(v)$ and different ideal decisions. Rename the pair so that $I_P(\omega(u)) = 1$ and $I_P(\omega(v)) = 0$. Since the monitor views are identical, determinism gives

$$M(\rho(u), P) = M(\rho(v), P) = d.$$

If $d = \text{ALLOW}$, soundness fails on v. If $d \in \{\text{DENY}, \text{ABSTAIN}\}$, permissiveness fails on u. Every possible decision violates at least one property, proving the theorem. □

Randomized monitors. The same obstruction applies to probability-one guarantees. Equal inputs induce the same output distribution. Assigning positive probability to ALLOW gives positive unsafe-allow probability on the unauthorized context; assigning probability one to non-allow decisions gives false denial or abstention with probability one on the authorized context. Randomization can trade the two errors but cannot eliminate both.

Proof of Proposition 3. The separating pair directly violates Definition 1. Theorem 2 then gives the downstream monitor consequence. □

Proof of Theorem 4. Suppose the initial representation has $c_0 \geq 1$ nonempty cells. Each successful counterexample-guided step splits at least one nonempty cell into two nonempty cells, and no later step merges cells. After k refinements there are at least $c_0 + k$ cells, while a partition of D has at most $|D|$ cells. There can therefore be at most $|D| - c_0 \leq |D| - 1$ such splits. If the stopping condition leaves no policy-mixed cell, then for every pair in that cell and every $P \in \mathcal{P}$ the ideal decisions agree. Equality of representations therefore implies equality of the full policy-decision vector, which is exactly authorization sufficiency on $(D, \mathcal{P})$. □

Field necessity. Let π_{-f} erase atom component f. Component f is necessary on $(D, \mathcal{P})$ if $\pi_{-f} \circ \rho_K$ is not authorization-sufficient although ρ_K is. Theorem 2 then shows that erasing f forces an unsafe allow or loss of permissiveness for some admissible policy. This is a relative necessity result: changing D or the admissible policy family can change which components are necessary.

Proof of Theorem 5. Let u be any allowed committed effectful call. Complete mediation gives a pre-commit decision over $A_{d_t}(u)$, and the authorizer's allow rule gives $J_P(A_{d_t}(u)) = 1$. Descriptor fidelity yields $I_P(\omega(u)) = 1$. Check–use consistency ensures that the context whose semantic commit view is $\omega(u)$ is exactly the checked context and that the same policy snapshot governs its commitment. □

Proof of Corollary 6. Consider an allowed and committed call. Descriptor soundness puts each of its effect-bearing values in $F(u)$. Provenance soundness puts every value introduced by untrusted evidence in $U_V(h)$, and grounding soundness excludes it from $G_V(q)$ unless the authenticated task independently supports it. The guard's first allow condition therefore rejects a solely untrusted value. If an untrusted instruction changes only a registered effect kind, provenance soundness places that kind in $E(u) \cap U_E(h)$ and grounding soundness excludes it from $G_E(q)$; the second allow condition rejects it. Thus no allowed registered effect or effect-bearing value is introduced solely by untrusted evidence. Complete mediation applies this argument to every effectful commit, and check–use consistency ensures the argument concerns the call actually executed. □

Scope of the proofs. The proofs do not establish descriptor, provenance, or grounding soundness. Counterfactual execution can falsify a descriptor and can establish sufficiency only on a finite collision-complete domain. It cannot certify unseen, asynchronous, or out-of-domain effects. The atom-mediation theorem additionally assumes policy correctness and policy-snapshot integrity; it does not establish either assumption. The confinement theorem covers only the implemented provenance-origin policy. ACLs, delegation, quotas, history-dependent policy, and authenticated malicious requests require additional policy machinery.